Guide to running a custom manual test

This guide explains how to run a single test with parameters that differ from the automated test matrices. It assumes that `network_monitor2` is already installed and configured on NUC1 and NUC2.

> **Safety:** do not modify VLAN27, `enp114s0`, the management addresses, or the default route. Always reach NUC1 through its management address, `10.10.27.10`.

1. Define the test

Before starting, select the path, traffic direction, protocol, port, duration, number of flows, packet size and either the TCP window size or, for UDP, the offered bandwidth.

Record the active path before the test so that you can restore it afterward:

```
ip route show 10.10.22.0/24
ssh 10.10.27.10 'ip route show 10.10.42.0/24'
```

Path	NUC2 IP	NUC1 IP
CEIT	`10.10.42.11`	`10.10.22.10`
INDRA	`10.10.42.11`	`10.10.22.10`
Bypass TN0	`10.10.28.11`	`10.10.28.10`
Bypass TN1	`10.10.29.11`	`10.10.29.10`

CEIT and INDRA use the same endpoints but different gateways.

2. Select the path

CEIT

From NUC2:

```
sudo /home/tuser/scripts/test_ceit
ssh -t 10.10.27.10 'sudo /home/tuser/scripts/test_ceit'

ip route get 10.10.22.10 from 10.10.42.11
ssh 10.10.27.10 'ip route get 10.10.42.11 from 10.10.22.10'
```

The first result must use gateway `10.10.42.41`; the second must use `10.10.22.40`.

INDRA

From NUC2:

```
sudo /home/tuser/scripts/test_indra
ssh -t 10.10.27.10 'sudo /home/tuser/scripts/test_indra'

ip route get 10.10.22.10 from 10.10.42.11
ssh 10.10.27.10 'ip route get 10.10.42.11 from 10.10.22.10'
```

The first result must use gateway `10.10.42.31`; the second must use `10.10.22.30`.

Bypass TN0 or TN1

Remove only the CEIT/INDRA test routes:

```
sudo ip route del 10.10.22.0/24 via 10.10.42.41 2>/dev/null || true
sudo ip route del 10.10.22.0/24 via 10.10.42.31 2>/dev/null || true

ssh -t 10.10.27.10 \
  'sudo ip route del 10.10.42.0/24 via 10.10.22.40 2>/dev/null || true;
   sudo ip route del 10.10.42.0/24 via 10.10.22.30 2>/dev/null || true'
```

Check the selected bypass:

```
# TN0
ip route get 10.10.28.10 from 10.10.28.11

# TN1
ip route get 10.10.29.10 from 10.10.29.11
```

The result must show a directly connected route with no gateway.

3. Check reachability

```
ping -I <NUC2_IP> -c 5 <NUC1_IP>
```

TN0 example:

```
ping -I 10.10.28.11 -c 5 10.10.28.10
```

If the result is `100% packet loss`, do not start the test. Check the routes, VLANs, and ARP first.

4. Select the direction

Without `--reverse`, traffic flows from the NUC2 client to the NUC1 server. With `--reverse`, the server generates traffic toward the client while client-side results are still stored in the NUC2 database.

Alternatively, you can swap the server and client roles if you want NUC1 to act as the client and record the test.

5. Start the server

In a terminal connected to NUC1:

```
ssh 10.10.27.10
network_monitor2 --server --bind-ip <NUC1_IP> --port <PORT>
```

TN0 example using port `5051`:

```
ss -ltn | grep -q ':5051 ' && echo 'PORT IN USE' || echo 'PORT AVAILABLE'
network_monitor2 --server --bind-ip 10.10.28.10 --port 5051
```

Leave the server running. If `Address already in use` appears, select another port and use it on both the server and client.

6. Build the custom command

```
network_monitor2 <NUC1_IP> \
  --source-ip <NUC2_IP> \
  --port <PORT> \
  --duration <SECONDS> \
  --tcp|--udp \
  [--bandwidth <RATE>] \
  [--packet-size <BYTES>] \
  [--window-size <SIZE>] \
  [--flows <1-32>] \
  [--reverse]
```

Parameter	Effect
`--duration`	Test duration; default is `30` seconds
`--tcp`	Run a TCP test
`--udp`	Run a UDP test; this is the default protocol
`--bandwidth`	Total offered UDP traffic, such as `75M` or `1G`
`--packet-size`	UDP datagram size (`iperf3 -l`) or TCP MSS (`iperf3 -M`); default `1460` bytes
`--window-size`	TCP socket buffer/window, such as `256K` or `1M` ; TCP only
`--flows`	Number of parallel flows, from 1 to 32; default is 1
`--reverse`	Send traffic from the NUC1 server to the NUC2 client

With UDP and multiple flows, the total bandwidth is divided automatically. For example, `--bandwidth 200M --flows 4` creates four `50M` flows.

The port must be between `1` and `65535` . For UDP, select the packet size with the path MTU in mind: oversized values can cause fragmentation and should be used only when fragmentation is part of the test.

If `--packet-size` is omitted, the tool still passes and stores the explicit default `1460` . New results therefore do not have `packet_size=NULL` . With an MTU of 1500, do not use a TCP MSS above `1460` .

With multiple TCP flows, `--window-size` applies to each flow. The operating system's maximum socket-buffer settings may limit the effective value.

Both NUCs are configured to accept a `1M` window size through 4 MiB socket-buffer ceilings. Before testing, verify them without changing the system:

```
cat /proc/sys/net/core/rmem_max
cat /proc/sys/net/core/wmem_max
```

Both values must be `4194304` on NUC1 and NUC2.

7. Custom examples

TCP from NUC2 to NUC1 for 45 seconds

```
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 45 --tcp
```

TCP from NUC1 to NUC2 with 8 flows and a 1400-byte MSS

```
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 45 --tcp \
  --reverse --flows 8 --packet-size 1400
```

TCP with a 64 KiB window size

```
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 60 --tcp \
  --window-size 64K
```

TCP with a 1 MiB window size

```
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 60 --tcp \
  --window-size 1M
```

UDP at 75 Mbit/s with 1000-byte datagrams

```
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 90 --udp \
  --bandwidth 75M --packet-size 1000
```

Reverse UDP at a total of 200 Mbit/s over four flows

```
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 60 --udp \
  --bandwidth 200M --flows 4 --reverse
```

For CEIT, INDRA, or TN1, change only `NUC1_IP` and `NUC2_IP` according to the table above.

The two requested tests

With the TN0 server running on port `5051`, run from NUC2:

```
# UDP at 500 Mbit/s: focus on receiver lost_percentage
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 120 --udp --bandwidth 500M

# TCP with a 1 MiB socket/window: focus on receiver throughput
network_monitor2 10.10.28.10 \
  --source-ip 10.10.28.11 \
  --port 5051 --duration 120 --tcp --window-size 1M
```

For UDP, the tool collects the server's receiver output. The `lost_percentage` stored in the DB, Excel, and Grafana therefore represents receiver loss rather than the zero reported by the sender.

8. Export only the completed test

In the same client terminal, capture the time immediately before the test:

```
TEST_START="$(date '+%F %T')"
```

Run the custom command and capture the end time as soon as it finishes:

```
TEST_END="$(date '+%F %T')"
printf 'Test interval: %s .. %s\n' "$TEST_START" "$TEST_END"
```

Build the filename from the same timestamps and export that exact interval:

```
TEST_START_FILE="$(date -d "$TEST_START" '+%Y%m%d_%H%M%S')"
TEST_END_FILE="$(date -d "$TEST_END" '+%Y%m%d_%H%M%S')"
EXPORT_FILE="custom_test_${TEST_START_FILE}_to_${TEST_END_FILE}.xlsx"

network_monitor2 --export-excel "$EXPORT_FILE" \
  --start "$TEST_START" \
  --end "$TEST_END"

printf 'Exported file: %s\n' "$EXPORT_FILE"
```

The resulting name, for example `custom_test_20260908_101500_to_20260908_101545.xlsx`, unambiguously identifies the test start and end times.

The exporter automatically applies the same suffix whenever both `--start` and `--end` are provided; if the filename already contains it, the suffix is not duplicated.